

# **3.(라즈베리파이)**

## **파이썬 기초**

**IoT 사물인터넷**

# 학습목표

---

- 파이썬을 사용하기 위한 환경을 설치한다.
- 파이썬 함수, 클래스 및 모듈을 이해한다.

# 1. 파이썬 시작하기

파이썬은 다양한 프로그램 중에 한가지 입니다.



인간의 언어만 번역을 할 때도 단어만 1대1로 번역을 한다고 해서 알아 들을 수 있는건 아니죠. 그렇기 때문에 프로그래밍 언어도 중간에 어떻게 번역을 해 주는지가 중요합니다.

파이썬이다 그러면 우리가 문법을 주로 생각하는데요 . 파이썬 문법을 번역해 주는 번역기 소프트웨어를 의미하기도 합니다.

컴퓨터의 구조나 작동원리는 변화지 않아요 하지만 어떤 프로그래밍 언어로 어떤일을 하는지에 따라서 프로그램의 효율성이 많이 달라집니다.

여러분이 일하는 자체도 많이 달라질 수 있어요 그렇기 때문에 소프트웨어 분야에서 프로그래밍 언어는 매우 중요합니다.

# 1. 파이썬 시작하기

---

## 2. 누가 처음에 만들었나요?



파이썬 언어의 창시자

“구이도 반 로섬”

Guido van Rossum

**파이썬은 네덜란드 프로그래머 구이도 반 로섬이 만들었습니다.  
2005년에 구글에서 일을 했었고 2022년도 부터는 MS 에서 일하고  
계신다고 합니다.**

# 1. 파이썬 시작하기

---

## 3. 왜 '비단뱀'인가요?



파이썬 로고



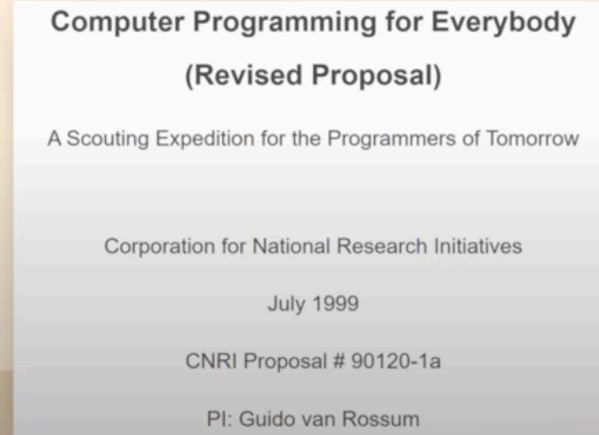
코미디 쇼  
“몬티 파이썬의 비행 써커스”

프로그래밍 언어에 왜 뱀 이름을 붙였나 이상하게 생각하실수도 있을것 같은데요  
자신이 좋아하던 코미디 쇼에서 가져온거라고 합니다. 이름을 지을때 짧고 독특하고  
신비로운 찾고 있었다고 하던데 의도대로 이름을 잘 지으신것 같습니다.

# 1. 파이썬 시작하기

## 4. 왜 만들었나요?

- 배우기 쉽고 생산성 높은 프로그래밍 언어

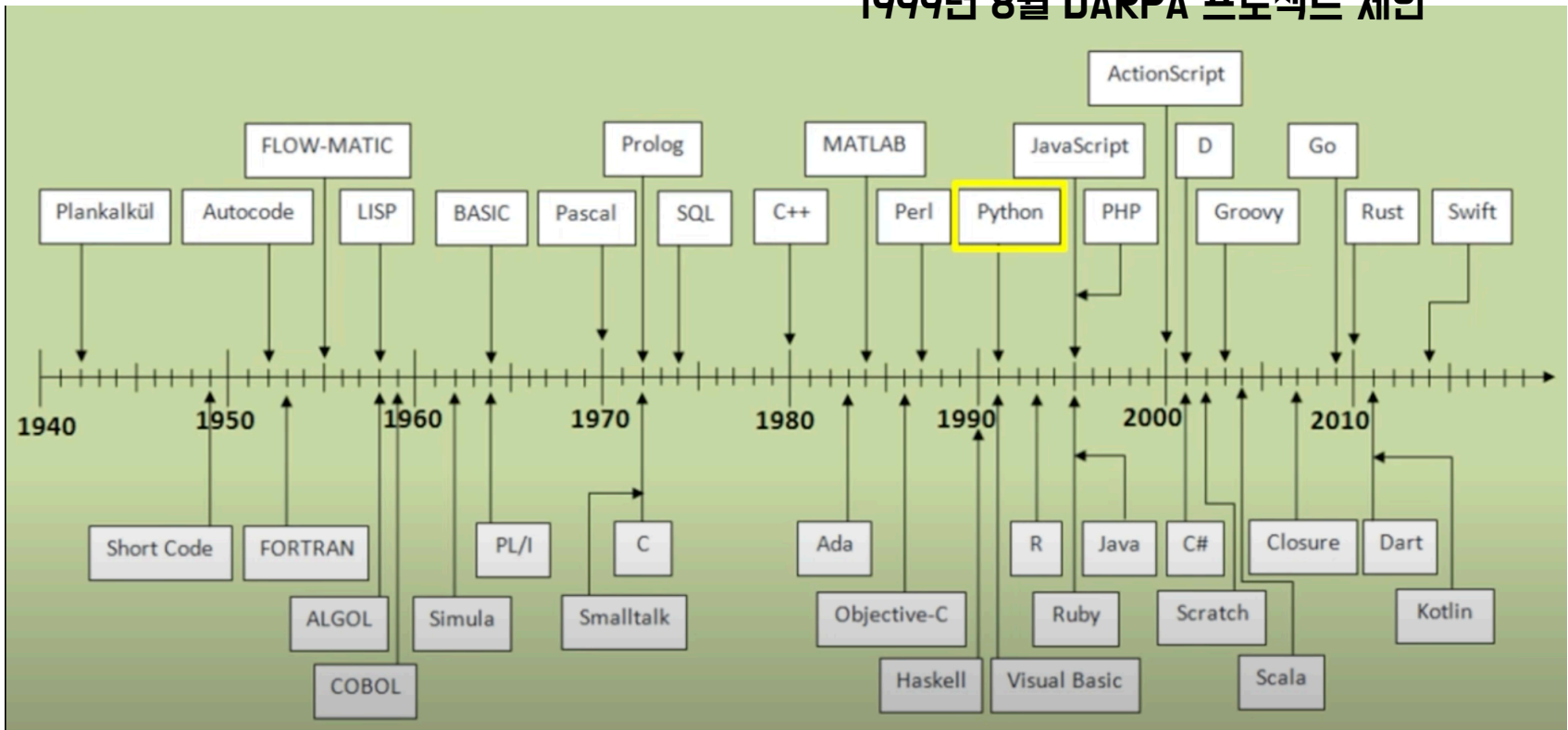


서양에서는 크리스마스 연휴동안 길게 쉬는 문화를 가지고 있는데요 이 분이 크리스마스 동안 취미로 할 프로젝트를 찾고 있었다고 합니다. 그런 얘기가 전설처럼 내려 오고 있긴 하지만 사실은 크리스마스 주중에 똑딱하고 만들어 낸건 아니구요 이미 예전부터 ABC 라는 언어를 작업을 하고 있었고 그 연장선상에서 파이썬이라는 언어를 만들어 내신 거예요 예초부터 파이썬의 목표는 고등학생이나 비전공자도 쉽게 배워서 편하게 쓸수 있는 언어를 만든것이었다고 합니다.

# 1. 파이썬 시작하기

## 5. 언제 만들어졌나요?

1991년 2월 0.9.0 버전 발표  
1999년 8월 DARPA 프로젝트 제안



지금 보시는 그림은 프로그램 언어들이 언제 만들어 졌는지 보여주는 그림인데요  
파이썬 주의를 보시면, Perl, PHP, JavaScript도 있네요. 많은 언어들이 탄생된 시기입니다.

# 1. 파이썬 시작하기

## 파이썬 1위

### 6. 인기가 많은 이유는 뭡가요?

| Feb 2022 | Feb 2021 | Change | Programming Language                                                                                 |  | Ratings | Change |
|----------|----------|--------|------------------------------------------------------------------------------------------------------|--|---------|--------|
| 1        | 3        | ▲      |  Python             |  | 15.33%  | +4.47% |
| 2        | 1        | ▼      |  C                  |  | 14.08%  | -2.26% |
| 3        | 2        | ▼      |  Java               |  | 12.13%  | +0.84% |
| 4        | 4        |        |  C++                |  | 8.01%   | +1.13% |
| 5        | 5        |        |  C#                 |  | 5.37%   | +0.93% |
| 6        | 6        |        |  Visual Basic       |  | 5.23%   | +0.90% |
| 7        | 7        |        |  JavaScript         |  | 1.83%   | -0.45% |
| 8        | 8        |        |  PHP                |  | 1.79%   | +0.04% |
| 9        | 10       | ▲      |  Assembly language |  | 1.60%   | -0.06% |
| 10       | 9        | ▼      |  SQL              |  | 1.55%   | -0.18% |

그림 1. 2022년 2월 TIOBE 프로그래밍 언어 순위 (출처: <https://www.tiobe.com/tiobe-index>).

- 파이썬은 여러가지 프로그래밍 인기 조사에서 1위를 하는 경우가 많습니다.



# 1. 파이썬 시작하기

---

## 6. 인기가 많은 이유는 뭐가요?

- 1) 파이썬은 문법이 간결해서 배우기 쉽고, 사용하기에도 쉬움
- 2) 파이썬은 개발 속도가 빠르다. (생산성 향상)
- 3) 파이썬은 커뮤니티가 발달되어 있다.
- 4) 파이썬은 다양한 응용 분야에서 활용되고 있다.



# 1. 파이썬 시작하기

---

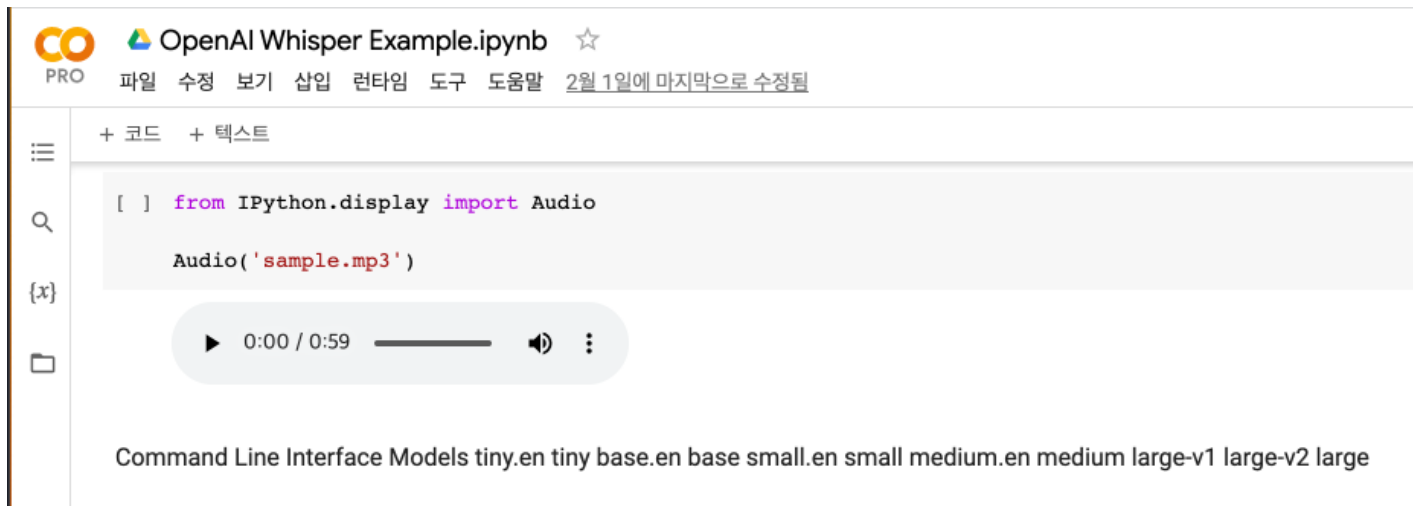
## 7. 파이썬도 단점이 있나요?

- 1) 전반적으로 속도가 느리다
- 2) 메모리를 낭비한다
- 3) 멀티코어 활용이 비효율적이다

## 2. 파이썬 설치하기

### 1. 여러가지 파이썬 설치 방법들

- 1) 직접설치
- 2) 아나콘다/미니콘다 설치
- 3) 구글 코랩 및 기타 온라인 환경사용

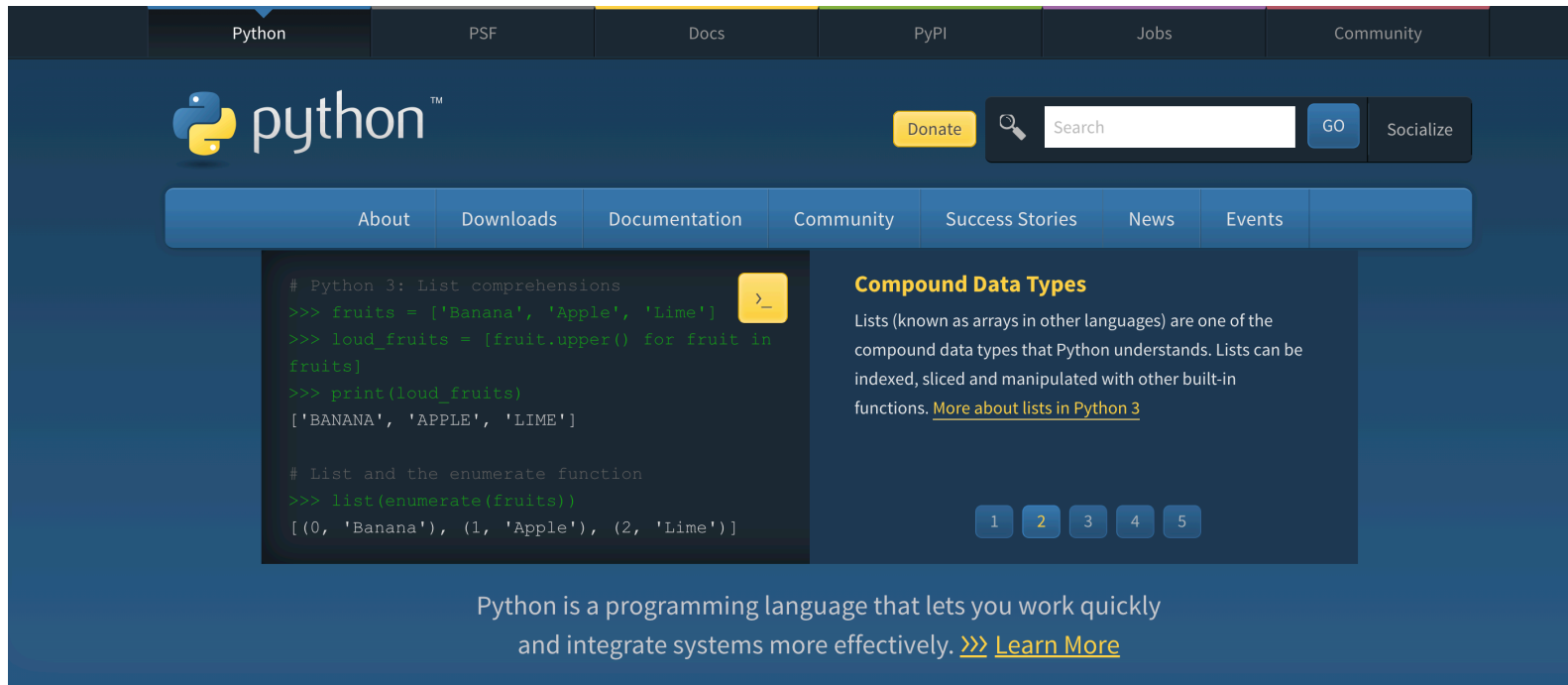


## 2. 파이썬 설치하기 (Windows 버전)

### 1. 파이썬 사이트

1) <https://www.python.org/>

2) Python 최신 버전 다운로드



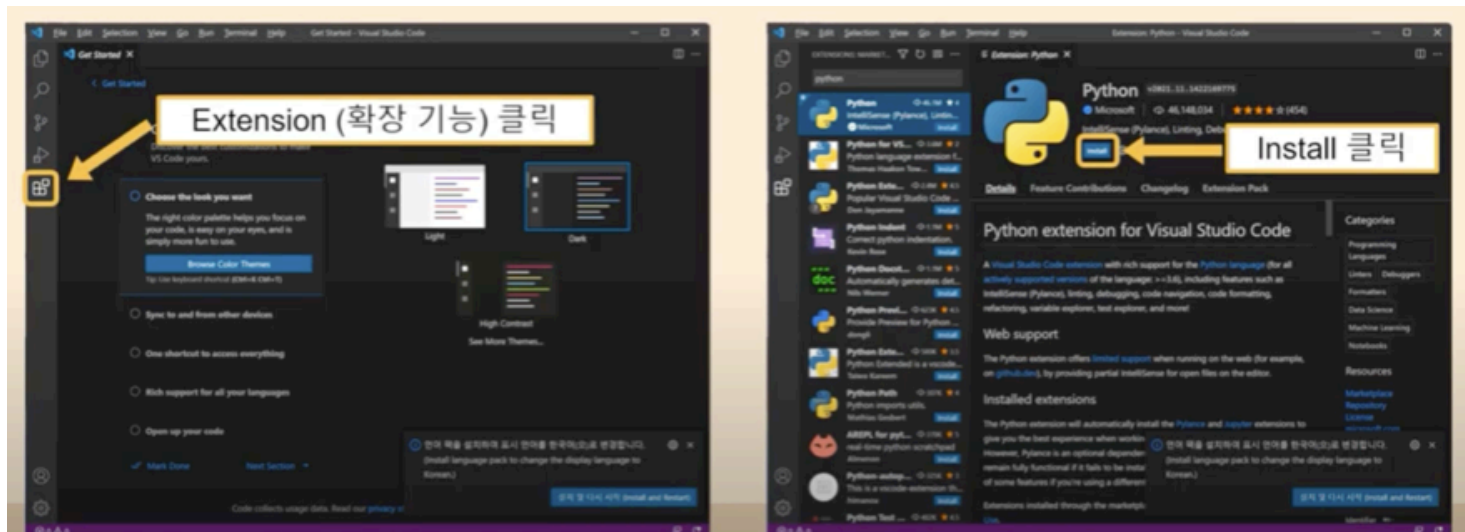
## 2. 파이썬 설치하기 (Windows 버전)

VSC

### 2. 비주얼 스튜디오 코드(Visual Studio Code)

- 1) 프로그래머를 위한 문서 편집기
- 2) 완전히 무료이며 가볍고 빠름
- 3) 다양한 운영체제와 언어 지원
- 4) 기능 확장이 쉽다.
- 5) 최근 사용자 층이 늘어나고 있는 IDE(통합개발환경)

### 3. 파이썬 확장기능 추가



# 3. 파이썬

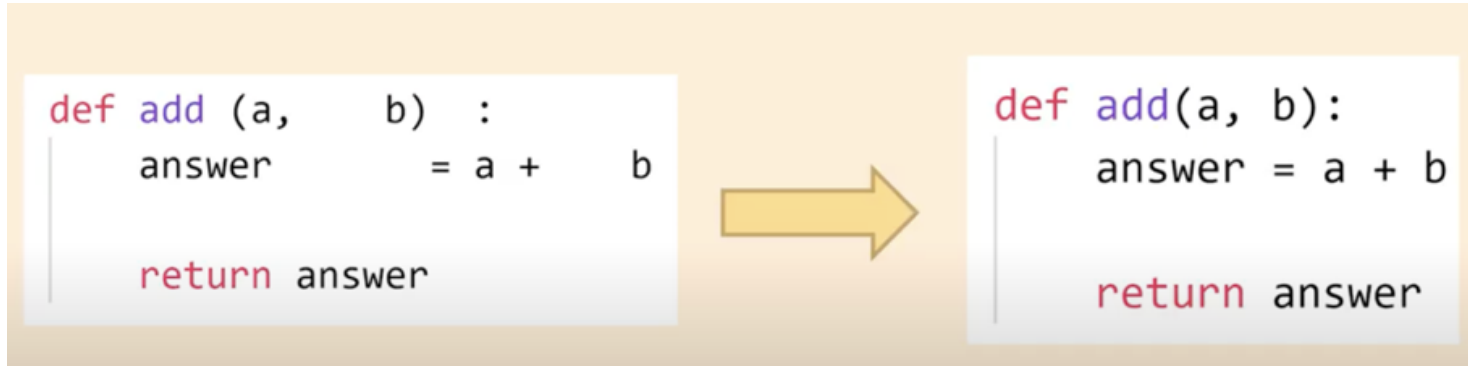
---

## 강의의 목적

- 실제로 프로그램을 만들 수 있는 능력
- 장기적으로 성장할 수 있는 튼튼한 기초
- 아이디어를 실현할 수 있다는 자신감

# 3. 파이썬 - 포맷터(Formatter)

black



## 포맷터 사용법

포맷터를 사용하면 빈칸 띄우기와 같은 사소한 규칙들은 신경쓰지 않아도 되기 때문에 편리합니다. 물론, 포맷터가 오류를 알아서 수정해 주지는 않습니다. 여기에서는 Black 이라는 포맷터를 사용합니다. VSCode확장으로 쉽게 설치할 수 있습니다.

1. VSCode의 좌측 하단 관리(톱니모양) 클릭 -> 설정 클릭
2. 위의 '설정검색'에서 python formatting provider 검색 후 **black**으로 설정(다른 포맷터도 선택 가능)
3. 위의 '설정검색'에서 format on save 검색 후 체크

를 하시면 .py파일을 저장할 때마다 포맷팅을 해줍니다.

# 3. 파이썬 - 자료형

---

## 1. 객체

파이썬에서는 모든 것이 객체입니다.  
메모리에 저장된 모든 데이터를 객체라는 개념을 통해서 처리합니다.

파이썬에서는 다양한 객체들을 사용하는데  
어떤 값을 가질 수 있느냐와  
무엇을 할 수 있느냐로 분류됩니다.

객체가 어떤 데이터 값으로 이루어져 있으며 어떤 행위를 할 수 있는지 정해놓은 것을  
자료형이라고 부릅니다.

파이썬에서는 다양한 자료형들을 기본적으로 제공해줍니다. 필요한 자료형(Data Type)  
들을 직접 만들어서 사용할 수도 있습니다.

예) 정수형 `int`, 실수형 `float`, 문자열 `str` 등



# 3. 파이썬 - 주식

---

## 1. 파이썬의 주식

```
# 1. 한줄 주석 : # (프로그램을 코딩 중 제일 중요한 단어나 급하게 필요할 때  
# 한줄 주석으로 '#'을 사용해줍니다.  
# 2. 여러줄 주석 : '''(홀 따옴표 3개를 사용), """(쌍 따옴표 3개를 사용)  
# 주석을 달아줄 때도 같이 혼, 쌍 따옴표 3개를 이용해 달아주면 됩니다.  
  
'''  
여러줄 주석1  
'''  
  
"""  
여러줄 주석2  
"""  
  
print('주석예제입니다.')
```

# 3. 파이썬 - 자료형

## 1. 숫자형 : 정수 및 실수를 표현하는 타입

```
1 a=123
2 print(type(a)) #type함수는 변수의 타입을 리턴
3 a=100*100
4 print(a)
5 a,b = 9,2
6 print(a*b)
7
8 #01:정수 a를 선언하고 123을 할당합니다.
9 #02:a가 어떤 타입인지 출력을 합니다.
10 #05:a는 9, b는 2로 각각 값을 할당합니다.
```

```
<class 'int'>
10000
18
```

## 2. 문자형 : 문자열을 표현하는 타입

```
1 a="파이썬 만세" #쌍 따옴표로 문자열 값을 선언
2 print(a)
3 print(type(a)) #type 함수는 변수의 타입을 리턴
4 b='python go' #홀 따옴표로 문자열 값을 선언
5 print(type(b))
6 print(b)
```

```
파이썬 만세
<class 'str'>
<class 'str'>
python go
```

# 3. 파이썬 - 자료형

---

## 2. 문자형 : split, join 사용해 보기

```
a = "a:b:c:d"
b = a.split(":")
print(b)  # ['a', 'b', 'c', 'd']
```

```
c = "#".join(b)
print(c)  # a#b#c#d
```

### 3. 파이썬 - 자료형

---

**[문제]** 전화번호,주민등록번호 split로 구분해 보세요

```
phoneNumber = "000-1111-2222"
```

```
address = "1234567-1234567"
```

### 3. 파이썬 - 자료형

#### 1. 리스트 : 여러 개의 값들을 하나의 변수로 묶어서 사용하는 타입

리스트는 파이썬에서 가장 유연하게 사용할 수 있고 많이 사용되는 컨테이너입니다.

| 이름  | type  | 예시                      | 가변성 | 순서유지 | 중복허용 |
|-----|-------|-------------------------|-----|------|------|
| 리스트 | list  | ["사과", 123, 3.14]       | 가변  | ○    | ○    |
| 튜플  | tuple | ("사과", 123, 3.14)       | 불변  | ○    | ○    |
| 집합  | set   | {"사과", 123, 3.14}       | 가변  | X    | X    |
| 사전  | dict  | {"원주율":3.14, 123:"일이삼"} | 가변  | 3.6+ | 키X값○ |

```
my_list = [] # 비어 있는 리스트
my_list = [1,2,3,4,5] # 아이템 나열
my_list = ["A string", 23, 100.32, "o"] # 다양한 자료형의 객체들을 담을 수 있음
my_list = [x+100 for x in range(1,11)] # 아이템들의 조건을 기술하여 선언
```

## 3. 파이썬 - 자료형

### 3. 리스트 : 여러 개의 값들을 하나의 변수로 묶어서 사용하는 타입

#### 1) 리스트를 사용하지 않고 숫자형 변수 4개를 선언하여 출력한 예

```
1 a,b,c,d=0,0,0,0
2 hap=0
3 a=int(input("1번째 숫자 : ")) #숫자를 입력받아 a에 그 값을 대입
4 b=int(input("2번째 숫자 : "))
5 c=int(input("3번째 숫자 : "))
6 d=int(input("4번째 숫자 : "))
7 hap=a+b+c+d
8 print("합계=>%d" % hap)    #%d는 % 뒤에 선언한 hap값이 대입
9
10 #01:4개의 숫자형 변수를 선언, 각 값들을 0으로 대입
11 #03~06:4개의 숫자를 입력받고, 변수 a,b,c,d에 대입
12 #07~08:4개의 값을 더해서 hap 출력
```

```
1번째 숫자 : 10
2번째 숫자 : 20
3번째 숫자 : 30
4번째 숫자 : 40
합계=>100
```

#### 2) 리스트 변수를 선언하여 앞 예제를 수정

```
1 aa=[0,0,0,0]    # 숫자 4개를 저장하는 리스트 변수
2 hap=0
3 aa[0]=int(input("1번째 숫자 : "))    #사용자가 입력한 숫자를
4 aa[1]=int(input("2번째 숫자 : "))    #리스트의 각 요소에 대입
5 aa[2]=int(input("3번째 숫자 : "))
6 aa[3]=int(input("4번째 숫자 : "))
7 hap=aa[0]+aa[1]+aa[2]+aa[3]
8 print("합계2 => %d" % hap)    #입력받은 4개의 값들을 합쳐서 콘솔창에 출력
9
10 #01:숫자 4개를 저장하는 리스트 변수 선언
11 #03~06:4개의 숫자를 입력받아서 리스트 변수 각각의 요소에 값을 대입
```

```
1번째 숫자 : 1
2번째 숫자 : 2
3번째 숫자 : 3
4번째 숫자 : 4
합계2 => 10
```

## 3. 파이썬 - 자료형

### 3. 리스트 : 여러 개의 값들을 하나의 변수로 묶어서 사용하는 타입

#### 3) 가변 리스트 사용

```
1 aa=[] #요소가 0개인 가변 리스트 변수 선언
2 aa.append(100) #가변리스트 aa에 값을 추가
3 aa.append(200)
4 aa.append(300)
5 aa.append(400)
6 #리스트 변수의 크기를 출력
7 print("리스트변수 크기는 %d" % len(aa))
8 print(aa) #요소의 값들을 출력
9 bb=[]
10 for i in range(0,100): #반복문 for문을 선언
11     bb.append(i) #초기값 0~99까지 반복
12 print(bb)
13
14 #02~05:요소의 값을 리스트 aa에 추가
15 #07:len함수를 사용하여 리스트 aa의 크기를 출력
16 #10~11:반복문 for를 이용하여 리스트 bb에 0~99까지 값을 추가
17 #12:리스트 bb에 저장된 값들을 출력
```

```
리스트변수 크기는 4
[100, 200, 300, 400]
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16,
17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31,
32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46,
47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61,
62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76,
77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91,
92, 93, 94, 95, 96, 97, 98, 99]
```

### 3. 파이썬 - 자료형

---

## `range( start, end)`

Range 함수는 start와 end 사이의 연속적인 숫자들을 리턴합니다.  
리스트를 만들거나 반복문에서 많이 사용

`rang( 0, 100 ) => 0, 1, 2, ..., 99`

`result = list(range(100))`



### 3. 파이썬 - 자료형

---

## `range( start, end, step)`

Range 함수는 start와 end 사이의 연속적인 숫자들을 step 간격으로 리턴합니다.  
리스트를 만들거나 반복문에서 많이 사용

`rang( 0, 100, 1 ) => 0, 1, 2, ..., 99`

`rang( 0, 100, 2 ) => 0, 2, 4, 8, ....., 98`

`rang( 0, 100, 3 ) => 0, 3, 6, 9, ....., 99`

**[문제]** `rang( 100, 0, -5 ) =>`

## 3. 파이썬 - 자료형

### 3. 리스트 : 여러 개의 값들을 하나의 변수로 묶어서 사용하는 타입

#### 3) 리스트의 위치 값 슬라이싱

```
1 aa=[10,20,30,40]      #리스트 변수 선언
2 #aa[-1]은 뒤에서 첫 번째, aa[-2]는 뒤에서 두 번째 요소를 의미
3 print("aa[-1]은 %d, aa[-2]는 %d" % (aa[-1],aa[-2]))
4 print(aa[0:2])         #첫 번째부터 두 번째 요소값을 출력
5 print(aa[2:4])         #세 번째부터 네 번째 요소값을 출력
6 print(aa[0:])          #첫 번째부터 끝까지 요소값을 출력
7
8 #03~05:리스트 aa의 범위를 지정하여 출력, 마지막 인덱스는 출력안함
```

```
aa[-1]은 40, aa[-2]는 30
[10, 20]
[30, 40]
[10, 20, 30, 40]
```

### 3. 파이썬 - 자료형

1. 리스트 : 여러 개의 값들을 하나의 변수로 묶어서 사용하는 타입
  - 2) 슬라이싱으로도 대입이 가능

```
my_list = [0, 1, 2, 3, 4, 5]
print(id(my_list))

my_list[1:3] = ["A", "B", "C"]
print(id(my_list))

print(my_list)
```

```
140636154004736
140636154004736
[0, 'A', 'B', 'C', 3, 4, 5]
```

### 3. 파이썬 - 자료형

---

#### 1. 리스트 : 여러 개의 값들을 하나의 변수로 묶어서 사용하는 타입

[문제]

```
my_list = [0, 1, 2, 3, 4, 5]
print(id(my_list))
```

```
my_list[1:5] = ["A", "B"]
print(id(my_list))
```

```
print(my_list)
```

```
[0, 'A', 'B', 5]
```

## 3. 파이썬 - 자료형

### 3. 리스트 : 여러 개의 값들을 하나의 변수로 묶어서 사용하는 타입

#### 4) 리스트에서 다양하게 사용 할 수 있는 함수

```
1 aa=[30,10,20]
2 print("현재의 리스트 : %s" % aa) #aa리스트 전체 출력은 문자열로 인식, %s사용
3 aa.append(40)
4 print("append 후의 리스트 : %s" %aa)
5 aa.pop()
6 print("pop 후의 리스트 : %s" %aa)
7 aa.sort()
8 print("sort 후의 리스트 : %s" %aa)
9 aa.reverse()
10 print("reverse 후의 리스트 : %s" %aa)
11 aa.insert(2,222)
12 print("insert(2,222) 후의 리스트 : %s" %aa)
13 print("20값의 위치 : %d" %aa.index(20))
14 aa.remove(222)
15 print("remove(222) 후의 리스트 : %s" %aa)
16 aa.extend([77,88,77])
17 print("extend([77,88,77]) 후의 리스트 : %s" %aa)
18 print("77값의 개수 : %d" %aa.count(77))
19
20 #03:append() 리스트 마지막에 값을 추가
21 #05:pop() 리스트 마지막 값을 꺼낸다.(삭제)
22 #07:sort() 리스트를 오름차순으로 정렬
23 #09:reverse() 리스트를 내림차순으로 정렬
24 #11:insert(i,j) 인덱스 i번째 위치에 j값을 삽입한다.
25 #13:index(i) 값i의 인덱스 번호를 반환
26 #14:remove(i) 값i를 리스트에서 삭제
27 #16:extend() 다른 리스트를 확장
28 #18:count(i) 리스트에서 i값의 갯수를 반환
```

현재의 리스트 : [30, 10, 20]  
append 후의 리스트 : [30, 10, 20, 40]  
pop 후의 리스트 : [30, 10, 20]  
sort 후의 리스트 : [10, 20, 30]  
reverse 후의 리스트 : [30, 20, 10]  
insert(2,222) 후의 리스트 : [30, 20, 222, 10]  
20값의 위치 : 1  
remove(222) 후의 리스트 : [30, 20, 10]  
extend([77,88,77]) 후의 리스트 : [30, 20, 10, 77, 88, 77]  
77값의 개수 : 2

### 3. 파이썬 - 자료형

---

#### 1. 리스트 : 여러 개의 값들을 하나의 변수로 묶어서 사용하는 타입 [문제]

```
my_list = ["Python", "Beyoun", "hello",  
           "compilers", "apple", "Apple"]  
  
my_list.sort()  
  
print(my_list)  
  
# 역순으로 정렬
```

### 3. 파이썬 - 자료형

1. 리스트 : 여러 개의 값들을 하나의 변수로 묶어서 사용하는 타입  
[따라해 보기] 리스트 + 기호 사용, extend 사용 차이점은?

```
a = [1, 2, 3]
print(id(a))

a = a + [4, 5]
print(id(a))
print(a)    # [1, 2, 3, 4, 5]

b = [1, 2, 3]
print(id(b))

b.extend([4, 5])
print(id(b))
print(b)    # [1, 2, 3, 4, 5]
```

## 3. 파이썬 - 자료형

### 3. 리스트 : 여러 개의 값들을 하나의 변수로 묶어서 사용하는 타입

#### 5) 다차원 리스트

```
1 aa=[[1,2,3,4],
2     [5,6,7,8],
3     [9,10,11,12]]
4 print(aa[0])      #aa 리스트의 첫 번째 리스트 전체 출력
5 print(aa[0][0])   #aa 리스트의 첫 번째 리스트의 첫 번째 요소 출력
6 print(aa[1][2])   #aa 리스트의 두 번째 리스트의 세 번째 요소 출력
7
8 #01~03:정수형 2차원 리스트를 선언
9 #      :다차원 리스트는 리스트 안에 리스트를 선언하는 구조
10 #04:첫 번째 리스트 전체를 출력하기 위한 코드
11 #05:첫 번째 리스트[0]에서 첫 번째 요소[0]
12 #06:두 번째 리스트[1]에서 세 번째 요소[2]
```

```
[1, 2, 3, 4]
1
7
```



### 3. 파이썬 - 자료형

1. 튜플 : 리스트와 유사한 기능. 리스트는 요소 값을 변경할 수 있지만 튜플은 한 번 값을 정하면 내부의 값을 바꿀 수 없다.

| 이름  | type  | 예시                      | 가변성 | 순서유지 | 중복허용 |
|-----|-------|-------------------------|-----|------|------|
| 리스트 | list  | ["사과", 123, 3.14]       | 가변  | O    | O    |
| 튜플  | tuple | ("사과", 123, 3.14)       | 불변  | O    | O    |
| 집합  | set   | {"사과", 123, 3.14}       | 가변  | X    | X    |
| 사전  | dict  | {"원주율":3.14, 123:"일이삼"} | 가변  | 3.6+ | 키X값O |

```
t = () # 비어있는 튜플
t = (1, 2, 3)
t = ("Hello", 3.14, 123) # 여러가지 객체들을 담을 수 있음
t = "hello", # 맨 뒤에逗가 있으면 아이템이 하나만 있는 튜플
t = ("hello",) # 포매터가 괄호를 넣어줌
t = 1, 2, 3
```

### 3. 파이썬 - 자료형

4. 튜플 : 리스트와 유사한 기능. 리스트는 요소 값을 변경할 수 있지만 튜플은 한 번 값을 정하면 내부의 값을 바꿀 수 없다.

#### 1) 문자열과 튜플 타입의 관계 - 1

```
1 str="파이썬 문자열"
2 print(str[0])
3 print(str[-1])
4 #str[-1] = '열' #뒤에서 첫번째 자리의 문자값 수정(오류)
5 card = 'red', 4, 'python', True #card 튜플을 선언
6 print(card) #card 튜플 전체 출력
7 print(card[1]) #card 튜플의 두 번째 요소 출력
8 #card[0] = 'blue' #card 튜플 첫 번째 요소 값 수정(오류)
9
10 #01~04: 문자열은 튜플타입의 변수로 선언 됨
11 #04 : 문자를 수정하려고 하면 에러 발생
```

파이썬  
(  
'red', 4, 'python', True)  
4

#### 2) 문자열과 튜플 타입의 관계 -2

```
1 one = "하나" #문자열 선언
2 print(type(one)) #one 변수의 타입 출력
3 one = '원' #문자열 값 전체를 변경
4 print(one) #수정된 문자열 출력
5 two = '둘' #문자열 선언
6 print(type(two)) #two 변수의 타입 출력
7 #two[0] = '투' #two 변수의 첫 번째 요소 값 변경(오류)
8 print(two[0]) #two 변수의 첫 번째 요소 값 출력
```

<class 'str'>  
원  
<class 'str'>  
둘

### 3. 파이썬 - 자료형

### 언패킹

4. 튜플 : 리스트와 유사한 기능. 리스트는 요소 값을 변경할 수 있지만 튜플은 한 번 값을 정하면 내부의 값을 바꿀 수 없다.

#### 3) 튜플의 분해 (언패킹)

```
1 card = 'red', 4, 'python', True      #card 튜플을 다양한 타입의 요소로 선언
2 a,b,c,d = card                      #card의 4개의 요소 값들을 a~d 변수에 할당
3 print(a)
4 print(b)
5 print(c)
6 d = False                          #변수 d의 값을 수정
7 print(d)
8
9 #01~02:4개의 다양한 타입의 요소가 있는 card 튜플변수를 선언하고
10 #   :각각의 요소 값들을 a,b,c,d변수에 각각 할당
11 #06:변수d는 튜플이 아님으로 값 수정이 가능
```

|        |
|--------|
| red    |
| 4      |
| python |
| False  |

**# 아이템의 개수가 맞지 않으면 오류가 발생합니다.**

```
a, b, c = card
ValueError: too many values to unpack
```

**# 아이템 중 일부를 사용하지 않을 예정이라면 언더스코어로 개수만 표시**

```
a, b, c, _ = card
```

### 3. 파이썬 - 자료형

---

**[문제]** 아래 코드의 실행 결과를 예측해 보세요.

```
my_tuple = (1,2,3)
```

```
print(my_tuple.index(1)) # 튜플 내 지정된 값이 몇 번째 인덱스인지
```

A. 0

B. 1

C. 2

D. ValueError

### 3. 파이썬 - 자료형

---

[문제] 튜플은 불변 컨테이너입니다. 다음 중 튜플 t에 대해서 사용할 수 없는 것을 골라 보세요.

t = (1,2,3)

A.t.pop()

B.sum(t)

C.t[:]

D.t.index(1)

### 3. 파이썬 - 자료형

#### 5. 집합 (set)

| 이름  | type  | 예시                      | 가변성 | 순서유지 | 중복허용 |
|-----|-------|-------------------------|-----|------|------|
| 리스트 | list  | ["사과", 123, 3.14]       | 가변  | O    | O    |
| 튜플  | tuple | ("사과", 123, 3.14)       | 불변  | O    | O    |
| 집합  | set   | {"사과", 123, 3.14}       | 가변  | X    | X    |
| 사전  | dict  | {"원주율":3.14, 123:"일이삼"} | 가변  | 3.6+ | 키X값O |

집합(set)은 시퀀스 자료형들과 많이 다릅니다. 예를 들어서 리스트 같은 경우는 아이টে을 넣은 순서를 그대로 유지해주고 같은 아이টে을 여러개 넣을 수도 있습니다.

그러나 집합(set)은 순서를 유지해 주지도 않고 같은 아이টে을 여러개 넣을 수도 없습니다.

# 3. 파이썬 - 자료형

---

## 5. 집합 (set)

`s = set()` # 빈 셋 만들 경우

리스트나 튜플과 달리 아이템의 순서를 유지해 주지 않습니다.

중복된 아이템들은 제거되고 하나만 남습니다. 인덱싱이나 슬라이싱을 사용할 수 없습니다.

`S = { 100, 55, 1, 1, 1, 1, 2, 3 }`

`print(S)` # {1, 2, 3, 55, 100}

`K = { 5, 6, 7 }`

`K.add(1)`

`print(K)` # { 1, 5, 6, 7 }

`#s = {} =>` 빈 셋을 만들려고 시도하면 `dict`가 만들어 짐, 빈셋은 `set()` 사용하세요.

### 3. 파이썬 - 자료형

---

#### 5. 집합 (set)

A = {0, 2, 4, 6, 8}

B = {1, 2, 3, 4, 5}

```
print("합집합 :", A | B) # A.union(B)
```

```
print("교집합 :", A & B) # A.intersection(B)
```

```
print("차집합 :", A - B) # A.difference(B)
```

```
print("대칭차집합 :", A ^ B) # A.symmetric_difference(B)    합집합-교집합
```

합집합 : {0, 1, 2, 3, 4, 5, 6, 8}

교집합 : {2, 4}

차집합 : {0, 8, 6}

대칭차집합 : {0, 1, 3, 5, 6, 8}



### 3. 파이썬 - 자료형

#### 6. 딕셔너리 : key값과 value값이 한 쌍으로 저장되는 타입

| 이름  | type  | 예시                      | 가변성 | 순서유지 | 중복허용 |
|-----|-------|-------------------------|-----|------|------|
| 리스트 | list  | ["사과", 123, 3.14]       | 가변  | O    | O    |
| 튜플  | tuple | ("사과", 123, 3.14)       | 불변  | O    | O    |
| 집합  | set   | {"사과", 123, 3.14}       | 가변  | X    | X    |
| 사전  | dict  | {"원주율":3.14, 123:"일이삼"} | 가변  | 3.6+ | 키X값O |

지금까지 배운 컨테이너들은 객체들이 나열되어 있는 형태였습니다.

사전(dict) 자료형은 두 객체가 키(key)와 값(value)으로 짝지어져서 나열되어 있는 형태입니다.

### 3. 파이썬 - 자료형

---

#### 6. 딕셔너리 : key값과 value값이 한 쌍으로 저장되는 타입

```
d = {}  
d = dict()  
d = {  
    "바나나": "외떡잎식물 생강목 파초과 바나나속에 속하는 식물의 총칭",  
    "아이언맨": "여심을 사로잡는 매력적인 미소의 백만장자 플레이보이 토니 스타크",  
    123: 456, # 마지막 컴마는 무시합니다.  
}
```

```
# 모두 같은 dict를 만듭니다.  
d1 = {"one":1, "two":2, "three":3}  
d2 = dict({"three":3, "one":1, "two":2})  
d3 = dict({"one":1, "three":3}, two=2)  
d4 = dict(one=1, two=2, three=3)
```

```
# 튜플의 리스트  
d5 = dict([("two", 2), ("one", 1), ("three", 3)])
```

```
# zip의 기능을 추측해 보세요.  
d6 = dict(zip(["one", "two", "three"], [1, 2, 3]))
```

### 3. 파이썬 - 자료형

---

#### 6. 딕셔너리 : key값과 value값이 한 쌍으로 저장되는 타입

**키가 중복될 경우에는** 마지막 하나만 남습니다.  
집합에서 아이템의 중복을 허용하지 않은 것과 비슷합니다.

```
d = {  
    "캡틴": "Captain",  
    "캡틴": "아메리카",  
}  
print(d) # {'캡틴': 아메리카 }
```

```
d = {  
    "A": 65,  
    "B": 66,  
    "C": 67,  
}  
print(d["B"]) # 66 대괄호안에 키를 넣어서 값을 찾을 수 있습니다.
```

### 3. 파이썬 - 자료형

---

#### 6. 딕셔너리 : key값과 value값이 한 쌍으로 저장되는 타입

```
d = {"번호": 10, "성명": "코난", "나이": 23, "사는곳": "서울"}  
print(d)
```

```
print(type(d)) # <class 'dict'>
```

```
print(d["나이"])
```

```
d["나이"] = 24 # 항목 변경  
print(d["나이"])
```

```
d["직업"] = "탐정" # key, value 추가  
print(d)
```

```
print(d.keys()) # dict_keys(['번호', '성명', '나이', '사는곳', '직업']) 키 값 전체 반환  
print(d.values()) # dict_values([10, '코난', 24, '서울', '탐정']) 값 전체 반환
```

```
print("사는곳" in d) # 키 값 존재 여부  
del d["사는곳"] # 키 값 삭제  
print("사는곳" in d)
```

```
print(d)
```

### 3. 파이썬 - 자료형

---

**[문제]** 다음 중 dict를 올바르게 정의한 것은?

A. `d={0.1*0.1 : '사과' }`

B. `d={[1] : "딸기" }`

C. `d= {1: "바나나" }`

**[힌트]** dict key 값이 float가 올 수 없습니다. (정밀도문제)

dict key 값이 리스트로 올 수 없습니다. (가변객체)

### 3. 파이썬 - 자료형

---

[문제] 다음 코드의 실행 결과를 예측해 보세요.

```
my_list = [1, 2, 3, 4, 5]  
del my_list[0:6:2]  
print(my_list)
```

- A. [1,3,5]
- B. [2,4]
- C. [1,2]

### 3. 파이썬 - 자료형

---

#### [따라해보기] 딕셔너리 값 추출하기

```
a = {"A": 90, "B": 80}
```

```
print(a["A"])
```

```
print(a["B"])
```

```
print(a["C"])
```

```
print(a.get("C"))
```

# 3. 파이썬 - 자료형

---

## 7. 표현식과 문장

### 표현식(Expression)

하나의 값으로 계산될 수 있는 리터럴, 변수, 연산자, 함수 호출 등의 조합을 의미합니다.

예시) `1.0 + abs(a) * b`

예시) `a > 1`

### 문장(Statement)

파이썬이 실행할 수 있는 명령을 의미합니다.

대입문 ( ex. `a = 1` ), 조건문, 반복문 등이 있습니다.

`print()` 같은 기능을 수행하는 함수 호출도 문장으로 볼 수 있습니다.



# 3. 파이썬 - 자료형

---

## 7. 표현식과 문장

### 블럭(Block)

파이썬에서는 들여쓰기 레벨이 동일한 문장들을 여러 줄을 묶어서 부를때 블럭이라고 부릅니다. 들여쓰기 덕분에 직관적으로 이해할 수 있습니다. 빈 줄은 무시합니다.

```
if 1 < a < 3:  
    print('1보다 큽니다.')  
    print('3보다 작습니다.')
```

# 3. 파이썬 - 자료형

## 8. 불리언(Boolean)

불리언 자료형의 객체는 True와 False 두 가지 값만을 가질 수 있습니다.  
문자열이 아니기 때문에 따옴표를 사용하지 않습니다.  
주로 조건문 if와 함께 흐름 제어를 할 때 많이 사용됩니다.

### 비교 연산자 (Comparison Operators)

아래 표의 예시는  $x = 1, y = 2$ 인 경우

| 연산자                | 설명                  | 예시                 |
|--------------------|---------------------|--------------------|
| <code>==</code>    | 값이 같은가?             | $(x == y)$ 는 False |
| <code>!=</code>    | 값이 다른가?             | $(x != y)$ 는 True  |
| <code>&gt;</code>  | 왼쪽 더 큰가?            | $(x > y)$ 는 False  |
| <code>&lt;</code>  | 오른쪽이 더 큰가?          | $(x < y)$ 는 True   |
| <code>&gt;=</code> | 왼쪽이 더 크거나 양쪽이 같은가?  | $(x >= y)$ 는 False |
| <code>&lt;=</code> | 오른쪽이 더 크거나 양쪽이 같은가? | $(x <= y)$ 는 True  |

[주의] 값이 같은지를 비교하는 연산자는 등호가 두 개입니다. 초보때 많이 실수합니다.

### 3. 파이썬 - 자료형

---

#### 8. 불리언(Boolean)

불리언 연산자 and

| 표현식             | 결과값   | 예시                       |
|-----------------|-------|--------------------------|
| True and True   | True  | (1 < 2 and 3 < 4)는 True  |
| False and True  | False | (1 > 2 and 3 < 4)는 False |
| True and False  | False | (1 < 2 and 3 > 4)는 False |
| False and False | False | (1 > 2 and 3 > 4)는 False |

### 3. 파이썬 - 자료형

---

#### 8. 불리언(Boolean)

불리언 연산자 or

| 표현식            | 결과값   | 예시                      |
|----------------|-------|-------------------------|
| True or True   | True  | (1 < 2 or 3 < 4)는 True  |
| False or True  | True  | (1 > 2 or 3 < 4)는 True  |
| True or False  | True  | (1 < 2 or 3 > 4)는 True  |
| False or False | False | (1 > 2 or 3 > 4)는 False |

### 3. 파이썬 - 자료형

---

#### 8. 불리언(Boolean)

불리언 연산자 not

| 표현식       | 결과값   | 예시               |
|-----------|-------|------------------|
| not True  | False | not 1 < 2는 False |
| not False | True  | not 1 > 2는 True  |

### 3. 파이썬 - 자료형

---

#### 8. 불리언(Boolean)

[문제] 다음 코드의 실행 결과는?

```
print( 1 < 2 or 3 < 2)
```

- A. True
- B. False

\* or은 좌우 둘 중 하나만 True이면 True입니다.

# 3. 파이썬 - 자료형

---

## 8. 불리언(Boolean)

[문제] 다음 코드의 실행 결과는?

```
a = [1]  
b = a.copy()
```

```
print( a == b, a is b )
```

- A. True True
- B. True False
- C. False True
- D. False False

\* ==는 값이 같은 지를 비교하고, is는 같은 객체인지(id가 같은지)를 비교합니다.  
\* print로 확인해 보세요.

```
print(id(a))  print(id(b))  
print(a)     print(b)
```

# 3. 파이썬 - 자료형

---

## 8. 불리언(Boolean)

[문제] 다음 코드의 실행 결과는?

```
print(bool([]))
```

- A. True
- B. False

\* 비어 있는 컨테이너는 False



# 3. 파이썬 - 제어문

## 1. if문

### 1) if, else, elif

```
1 a=23
2
3 #if문
4 if a<50:
5     print('50보다 작군요')
6
7 #if else문
8 if a<20:
9     print('20보다 작군요')
10 else:
11     print('20보다 크군요')
12
13 #elif문
14 age=int(input('현재 나이를 입력합니다.')) #사용자가 입력한 값을 정수로 리턴
15 if age < 10:
16     print('유년층 입니다.')
17 elif age < 20:
18     print('10대입니다.')
19 elif age < 30:
20     print('20대입니다.')
21 elif age < 40:
22     print('30대입니다.')
23 else:
24     print('장년층입니다.')
25
26 #다른 언어들은 중괄호를 사용하여 블록을 구분하지만
27 #파이썬은 들여쓰기를 통해서 블록을 구분(띄어쓰기 4칸)
```

50보다 작군요  
20보다 크군요  
현재 나이를 입력합니다.27  
20대입니다.

# 3. 파이썬 - 제어문

## 2. for문

### 1) for문을 선언하는 다양한 형태

|    |                                                      |            |
|----|------------------------------------------------------|------------|
| 1  | for i in range(0, 5, 1): #변수 i의 초기값은 0, 범위는 5, 증가값 1 | 0          |
| 2  | print(i) #조건이 맞는동안 print()실행                         | 1          |
| 3  | print("-----")                                       | 2          |
| 4  | for j in [1,3,5,7,9]: #for문을 처리할 목록에 리스트 변수 선언       | 3          |
| 5  | print(j)                                             | 4          |
| 6  | print("-----")                                       | -----      |
| 7  | for k in range(0,3,1):                               | 1          |
| 8  | print("꿈은 이루어 진다.")                                  | 3          |
| 9  |                                                      | 5          |
| 10 | #01~02:for문을 선언하고 i는 반복에 필요한 변수 값                    | 7          |
| 11 | # :range함수의 매개변수 중 첫 번째는 변수의 시작 값,                   | 9          |
| 12 | # :두 번째는 변수의 조건값, 5는 포함되지 않는다. 4까지                   | -----      |
| 13 | # :세 번째는 증감값, 조건이 맞을 때까지 반복을 하면서 i값 출력               | 꿈은 이루어 진다. |
| 14 | #04:for문을 선언할 때 처리할 목록은 리스트, 튜플, 문자열이 가능             | 꿈은 이루어 진다. |
| 15 | # :변수들의 첫 요소부터 마지막 요소까지 변수에 대입되어 반복                  | 꿈은 이루어 진다. |

# 3. 파이썬 - 제어문

## 2. for문

### 2) for문을 이용한 1부터 10까지 합을 구하는 예제

```
1 #for문을 이용하여 1에서 10까지 합을 구하시오.
2 sum=0
3 for i in range(1, 11, 1):    #초기값은 1, 증감값은 1, 조건값 11로 for문 선언
4     sum += i                #i변수값을 sum변수에 더함
5     print("sum : %d" %sum)   #1~10까지의 합을 출력
6     print("-----")
7 #for문을 이용하여 1에서 10까지 식과 합을 구하시오.
8 sum=0                        #sum변수를 0으로 초기화
9 for j in range(1, 11, 1):    #초기값은 1, 증감값은 1, 조건값 11로 for문 선언
10     if j<10:                #j가 10보다 작은 조건으로 if문 선언
11         print("%d + " %j, end="")
12     elif j==10:             #j가 10인 조건으로 if문 선언
13         print("%d = " %j, end="")
14     sum+=j
15
16 print("%d" %sum)            #1~10까지의 합을 출력
17
18 #11,13:end=""를 작성하면 자동 줄바꿈을 하지 않고 이어서 출력
sum : 55
-----
1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 = 55
```

# 3. 파이썬 - 제어문

## 3. while문

### 1) while문 선언과 반복문 안의 조건문 예제

```
1 str="꿈은 이루어 진다."  
2 i=0  
3 while i<3:  
4     print(str)  
5     i += 1  
6 print("-----")  
7 #while문으로 입력한 숫자만큼 str을 반복 출력하시오.  
8 i=int(input("반복 횟수 숫자를 입력합니다. "))  
9 j=0  
10 flag = True  
11 while flag:  
12     if i<=j:  
13         flag=False  
14     else:  
15         print(str)  
16         j=j+1  
17  
18 #반복문 조건문의 블록구분 (들여쓰기)
```

```
꿈은 이루어 진다.  
꿈은 이루어 진다.  
꿈은 이루어 진다.  
-----  
반복 횟수 숫자를 입력합니다. 5  
꿈은 이루어 진다.  
꿈은 이루어 진다.  
꿈은 이루어 진다.  
꿈은 이루어 진다.  
꿈은 이루어 진다.
```

# 3. 파이썬 - 제어문

## 4. break문

### 1) break를 사용하여 반복문을 빠져나오는 예제

```
1 #for문과 break문을 이용하여 1에서 20까지 합하다 100보다 작고 가장 가까운 합을 구하시오.
2 sum = 0
3 for i in range(1,50,1):
4     sum += i
5     if sum>100:
6         break; #break 뒤에 ;을 사용
7 sum = sum - i #100이 넘어가지 전의 값을 구하기 위하여 i를 뺀다.
8 print("%d" %sum)
9 print("-----")
10 #while문과 break문을 이용하여 1부터 입력한 숫자까지의 합을 구하시오
11 sum, i = 0, 1
12 j=int(input("숫자를 입력합니다. "))
13 while True:
14     if i<=j:
15         sum += i
16         i=i+1
17     elif j<i:
18         break;
19 print("1에서 %d까지의 합은 %d입니다." %(j, sum))
```

91

-----

숫자를 입력합니다. 5  
1에서 5까지의 합은 15입니다.

### 3. 파이썬 - 제어문

---

문제) 다음 코드의 실행 결과는?

```
for i in range(2, 6, 3):  
    print(i, end='')
```

A. 2345

B. 236

C. 25

\* range(start, stop, step)의 의미를 확인해 두세요.

### 3. 파이썬 - 제어문

---

문제) 다음 코드의 실행 결과는?

```
for i in range(2, 6, -2):  
    print(i, end='')
```

- A. 24
- B. 246
- C. 64
- D. 642
- E. 출력없음

\* step이 음수일 때는 start가 stop보다 커야 루프가 의미가 있습니다.

### 3. 파이썬 - 제어문

---

문제) 다음 코드의 실행 결과는?

```
my_list = [1, 2, 3]
```

```
For i in my_list:
```

```
    i *= 10
```

```
print(my_list)
```

A. [1,2,3]

B. [10,20,30]

C. 오류발생



### 3. 파이썬 - 제어문

---

문제) 다음 코드의 실행 결과는?

```
for i in range(5):  
    if i == 3:  
        break  
    print(i, end= ' ')
```

A. 0 1 2

B. 0 1 2 4

C. 0 1 2 3 4

D. 0 1 2 3 4 5

### 3. 파이썬 - 제어문

---

문제) A 학생의 50점 이상의 총합을 구하세요. (while, A.pop()함수 사용)

```
A = [20, 55, 67, 82, 45, 33, 90, 87, 100, 25]
```

```
sum = 0
for score in A:
    if score >= 50:
        sum += score

print(sum) #481
```

### 3. 파이썬 - 제어문

---

문제) A 학생의 50점 이상의 총합을 구하세요. (while, A.pop()함수 사용)

```
A = [20, 55, 67, 82, 45, 33, 90, 87, 100, 25]
```

```
sum = 0
while A:
    score = A.pop()
    if score >= 50:
        sum += score

print(sum)  # 481
```

### 3. 파이썬 - 제어문

---

문제) 숫자의 총합 구하기 (split)

```
user_input = "65,45,2,3,45,8"
```

### 3. 파이썬 - 제어문

---

문제) 사용자로부터 2~9의 숫자 중 하나를 입력받아 해당 숫자의 구구단을 한줄로 출력하는 프로그램을 작성하세요.

실행 예)

구구단을 출력할 숫자를 입력하세요(2~9) : 2  
2 4 6 8 10 12 14 16 18

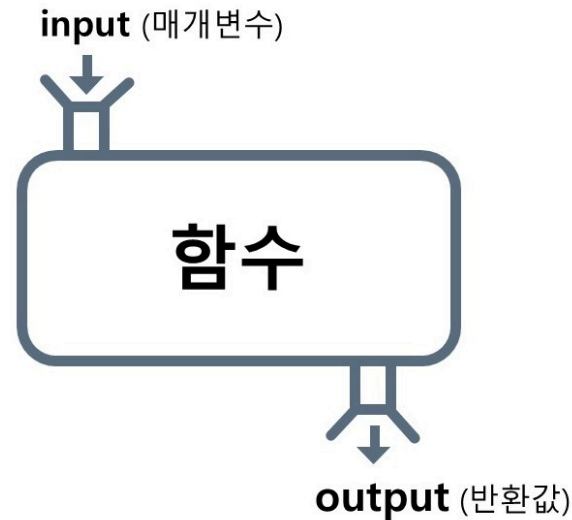
```
user_input = input("구구단을 출력할 숫자를 입력하세요(2~9):")
```

# 3. 파이썬 - 함수

---

## 1. 함수

$$y = f(x)$$



### 함수의 기본적인 사용법

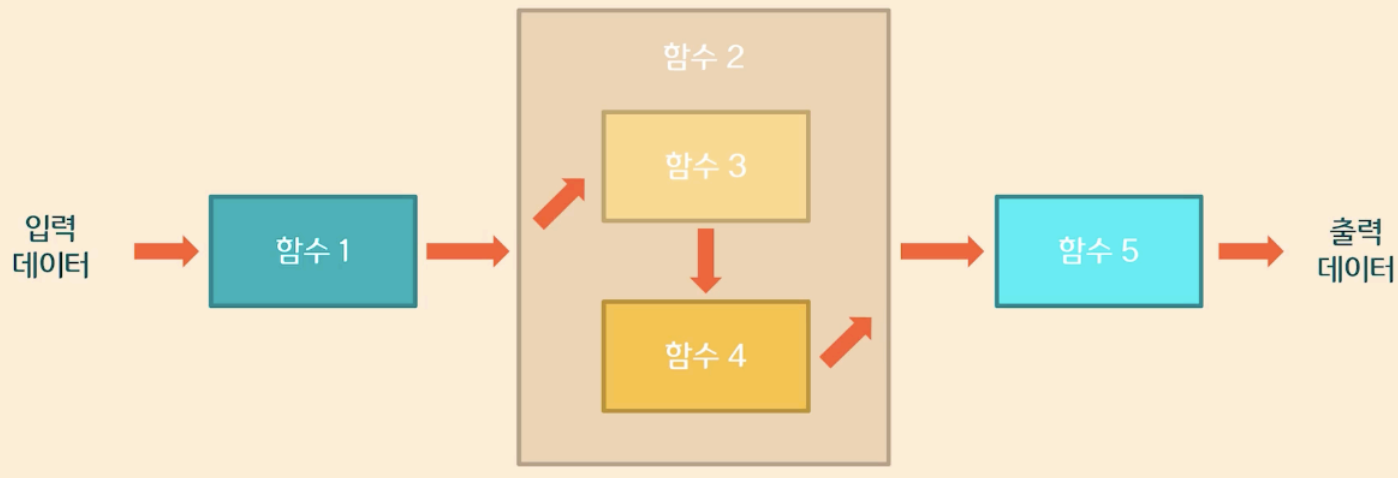
함수는 필요할 때만 불러서 사용할 수 있도록 코드를 묶어놓은 것입니다.  
코드를 재사용하기 편리하도록 '기능'별로 묶어서 분류해놓는 경우가 많습니다.

수학의 함수와 비슷한 점은 함수에 들어가는 입력과 나오는 출력을 항상 생각해  
보아야한다는 점입니다.

### 3. 파이썬 - 함수

#### 1. 함수

## 여러 함수들 사이의 데이터 흐름



크고 복잡한 문제를 작고 단순한 문제로 바꾸려면  
여러개로 쪼개서 생각하면 단순해 질 수 있습니다.

# 3. 파이썬 - 함수

## 1. 함수 선언하기

### 1) 함수를 선언하고 호출하는 예제

```
1 def mydef01():          #인수없는 함수 선언
2     print("함수를 선언합니다.") #들여쓰기를 통해 함수 구현 부분
3 def mydef02(str="인수함수를 선언합니다."): #인수 있는 함수 선언
4     print(str)          #인수로 받은 변수 출력
5
6 mydef01()              #함수 호출
7 mydef02()
8 mydef02("인수를 넣습니다.")
9
10 #01~02:인수 없는 함수를 선언
11 #03~04:인수 str을 가지고 있는 함수 선언, 기본값으로 문자열 사용
12 #07:인수없이 호출하면 기본값으로 출력
13 #08:문자열 인수를 넣어 호출하면, 입력한 문자열 값으로 출력
```

```
함수를 선언합니다.
인수함수를 선언합니다.
인수를 넣습니다.
```



### 3. 파이썬 - 함수

#### 1. 함수 선언하기

##### 2) 여러 개의 인수를 입력 받아서 계산하는 함수 예제

```
1 def mydef01():
2     i=10
3     j=20
4     print(i+j)
5
6 def mydef02(i, j):
7     print(i+j)
8
9 def mydef03(i, j, p):
10    if p=='+':
11        print(i+j)
12    elif p=='-':
13        print(i-j)
14    elif p=='*':
15        print(i*j)
16    elif p=='/':
17        print(i/j)
18
19 mydef01()      #함수 호출
20 mydef02(10,20) #함수 호출
21
22 n=int(input("첫번째 숫자를 입력합니다. "))
23 m=int(input("두번째 숫자를 입력합니다. "))
24 p=input("연산자를 입력하세요(+, -, *, /) ")
25 mydef03(n, m, p) #위에서 n, m, p인수값을 받아서 함수 호출
```

```
30
30
첫번째 숫자를 입력합니다. 30
두번째 숫자를 입력합니다. 10
연산자를 입력하세요(+, -, *, /) *
300
```

# 3. 파이썬 - 함수

---

## 1. 함수 선언하기

### 3) 키워드인수 (keyword argument)

```
def repeat_print(message, count):  
    """message를 count번 출력"""  
  
    for _ in range(count):  
        print(message)  
  
# 키워드 인수를 사용하면  
# 매개변수가 정의된 순서와 인수를 넣어주는 순서가 달라도  
# 오류가 발생하지 않아요  
repeat_print(count=3, message="안녕")
```

# 3. 파이썬 - 함수

## [문제]

pyramid draw 함수 만들기

파라미터로 받은 수로 pyramid 층을 구성

```
num_lines = 4

for j in range(1, num_lines + 1):
    for i in range(num_lines - j):
        print(" ", end="")

    for i in range(j * 2 - 1):
        print("*", end="")

    print()

    *
   ***
  *****
 *****
```

[해석]

```
000*
00***
0*****
*****
```

```
def draw_pyramid(num_lines):
    # 코드를 넣어 주세요
```

```
draw_pyramid(2)
draw_pyramid(4)
draw_pyramid(10)
```

### 3. 파이썬 - 함수

0 1 1 2 3 5 8 13 21

#### [따라해보기] 피보나치 함수

첫 번째 항의 값이 0이고 두 번째 항의 값이 1일 때, 이후에 이어지는 항은 이전의 두 항을 더한 값으로 이루어지는 수열을 피보나치 수열이라고 한다.

0 1 1 2 3 5 8 13 21 34 ...

```
def fib(n):  
    if n == 0:  
        return 0  
    if n == 1:  
        return 1  
    return fib(n - 2) + fib(n - 1)
```

```
print( fib(9) ) # 34
```

**[문제]** `print ( fib(40) )`

### 3. 파이썬 - 함수

---

#### [따라해보기] 파일 읽기/파일 저장

```
fruits = []
fruits.append("Guava")
fruits.append("Durian")
fruits.append("Strawberry")
fruits.append("PineBerry")
fruits.append("Lime")

f = open("fruit.txt", "w") # w 쓰기모드
for fruit in fruits:
    f.write(fruit)
    f.write("\n")
f.close()
```

### 3. 파이썬 - 함수

---

#### [따라해보기] 파일 읽기/파일 저장

```
f = open("fruit.txt", "r") # r 읽기모드
fruits = f.readlines()
f.close()

print(fruits) #['Guava\n', 'Durian\n', 'Strawberry\n', 'PineBerry\n', 'Lime\n']
```

### 3. 파이썬 - 함수

---

#### [따라해보기] “\n” 제거하기

```
r_fruits = []
for f in fruits:
    r_fruits.append(f.strip())
    #r_fruits.append(f.replace("\n", "")) #다른방법

print(r_fruits)

# 다른 방법
fruits = list(map(lambda s: s.strip(), fruits))
print(fruits)
```

### 3. 파이썬 - 함수

**[문제]** sample.txt 파일을 읽어 총합과 평균을 구한 후  
평균값을 result.txt 파일에 저장하는 프로그램을 작성하세요.

```
f = open("sample.txt")
lines = f.readlines()
f.close()

# 코드를 작성하세요.

f = open("result.txt", "w")
f.write(str(average))
f.close()
```

sample.txt

```
70
60
55
75
95
90
80
80
85
100
```



# 3. 파이썬 - 함수

## [알아두기] 오류와 예외처리

# 없는 파일을 읽을 경우 오류 발생

```
f = open("sample1.txt")  
lines = f.readlines()  
f.close()
```

FileNotFoundError: [Errno 2] No such file  
or directory: 'sample1.txt'

```
try:  
    f = open("sample1.txt")  
    lines = f.readlines()  
except FileNotFoundError:  
    print("파일이 없습니다. 파일명을 확인하세요")  
finally:  
    f.close()
```

sample.txt

```
70  
60  
55  
75  
95  
90  
80  
80  
85  
100
```

### 3. 파이썬 - 함수

---

#### [알아두기] 시간 읽어와서 파일명저장하기

```
import time

filename = time.strftime("%Y%m%d_%H%M%S")
print(filename)

f = open(filename + ".txt", "w") # w 쓰기모드
for fruit in fruits:
    f.write(fruit)
    f.write("\n")
f.close()
```

### 3. 파이썬 - 모스부호 해독하기

---

```
morse_dict = {  
    ".-": "A",  
    "-.-": "B",  
    "-.-": "C",  
    "-..": "D",  
    ".": "E",  
    ".-.-": "F",  
    "--." : "G",  
    "...": "H",  
    ".-.-": "I",  
    ".---": "J",  
    "-.-": "K",  
    "-.-": "L",  
    "--" : "M",  
    "-." : "N",  
    "----": "O",  
    ".---": "P",  
    "--.-": "Q",  
    "-.-": "R",  
    "...": "S",  
    "-": "T",  
    ".-.-": "U",  
    "...-": "V",  
    "--": "W",  
    "-.-.-": "X",  
    "-.-.-": "Y",  
    "--.-": "Z",  
    " " : " ",  
}
```

### 3. 파이썬 - 모스부호 해독하기

```
def morse_to_english(morse_text):  
    english_text = ""  
    morse_words = morse_text.strip().split(" ") # split the text into words  
    for morse_word in morse_words:  
        morse_chars = morse_word.split(" ") # split the word into characters  
  
        for morse_char in morse_chars:  
            if morse_char in morse_dict:  
                english_text += morse_dict[morse_char]  
            else:  
                # ignore characters that are not in the dictionary  
                pass  
        english_text += " " # add space between words  
    return english_text  
  
english_text = morse_to_english("-.--- --- ..-   .- -. .   .-   --. . -. . .-  
...")  
print(english_text)
```

### 3. 파이썬 - 모스부호 해독하기

---

**모스부호는**

**짧은 발신 전류(.)와  
긴 발신 전류(-)를 적절히 조합하여**

**알파벳과 숫자를 표기한 것입니다.**

**미국의 발명가 새뮤얼 핀리 브리즈 모스가 고안하여 '모스'라는 이름이 붙여졌습니다.**

### 3. 파이썬 - 모스부호 해독하기

---

```
[ '-.---',  
  '.-.-.',  
  '.-',  
  '--. .-..- ...']
```

4 words

첫번째

```
['-.-', '-.-', '..-']
```

3 chars

### 3. 파이썬 - 모스부호 해독하기

---

```
['', '.-', '-.', '.']  
4 chars
```

```
['', '.-']  
2 chars
```

```
['', '--.', '. ', '-.', '.. ', '..-', '...']  
7 chars
```

### 3. 파이썬 - 모스부호 해독하기

---

```
english_dict = {  
    "A": ".-",  
    "B": "-...",  
    "C": "-.-.",  
    "D": "-..",  
    "E": ".",  
    "F": ".-.-",  
    "G": "--.",  
    "H": ".-.-",  
    "I": "..",  
    "J": ".---",  
    "K": "-.-",  
    "L": "-.-.",  
    "M": "--",  
    "N": "-.",  
    "O": "---",  
    "P": ".---",  
    "Q": "--.-",  
    "R": "-.-",  
    "S": "...",  
    "T": "-",  
    "U": "-.-",  
    "V": "...-",  
    "W": "-.-",  
    "X": "-.-.-",  
    "Y": "-.-.-",  
    "Z": "--.-",  
    " ": " ",  
}
```



### 3. 파이썬 - 모스부호 해독하기

---

```
def english_to_morse(english_text):  
    morse_text = ""  
    for char in english_text:  
        if char.upper() in english_dict:  
            morse_text += english_dict[char.upper()] + " "  
        else:  
            # ignore characters that are not in the dictionary  
            pass  
  
morseText = english_to_morse("coding time")  
print(morseText)
```

### 3. 파이썬 - 클래스

#### 클래스의 정의 (객체를 정의하는 설계도)

- 클래스 이름
- 메서드
- 데이터속성(attributes)

self는 메서드를 호출하는 객체

특수한 메서드 `__init__`

- . Dunder(Double UNDERscores)
- . 객체(인스턴스)를 만들 때 자동으로 호출
- . C++/Java 생성자와 비슷

```
class MyInt:
    def __init__(self, value):
        self.v = value

    def add(self, a):
        self.v = self.v + a
```

```
i1 = MyInt(123)
print(i1.v)
```

```
i2 = MyInt(567)
print(i2.v)
```

```
i1.add(1)
print(i1.v)
```

```
i2.add(1)
print(i2.v)
```

### 3. 파이썬 - 클래스

클래스 예제를 더 추가해 주세요

1. 클래스 : 객체를 정의하는 설계도, 객체의 상태를 정의할 수 있는 속성과 객체의 기능을 정의하는 메서드를 가진다

```
class Automobile:
    def __init__(self, str):
        self.name = str

    def velocityPlus(self):
        self.velocity = self.velocity + 10

    def velocityDown(self):
        self.velocity = self.velocity - 10

    if self.velocity < 0:
        self.velocity = 0

ac = Automobile("푸조")
ac.velocity = 200
ac.velocityPlus()
print(ac.name + "의 속도는 %d 입니다." % ac.velocity)
```

### 3. 파이썬 - 클래스

---

[실습] 원 클래스 만들기

클래스 이름은 Circle

인스턴스 변수는 x,y,r 세가지 (x,y는 원의 중심의 좌표, r은 반지름)

`__init__()` 메서드와 `is_inside()` 메서드 구현

[힌트] 어떤 좌표 (i, j)에 대해서  $(i - x)^2 + (j - y)^2 < r^2$   
를 만족한다면 (i,j)가 원 안에 포함된 것으로 판단.

```
# 중심이 (1.0, 1.5)이고 반지름이 0.8인 원
my_circle = Circle(1.0, 1.5, 0.8)

print(my_circle.is_inside(1.5, 0.9)) # True
print(my_circle.is_inside(0.4, 0.5)) # False
```

### 3. 파이썬 - 클래스

---

[실습] 사칙연산 계산기

다음과 같이 동작하는 클래스 Calculator를 작성하시오

```
cal1 = Calculator([1, 2, 3, 4, 5])
print(cal1.sum()) # 15
print(cal1.avg()) # 3.0

cal2 = Calculator([6, 7, 8, 9, 10, 11])
print(cal2.sum()) # 51
print(cal2.avg()) # 8.5
```

### 3. 파이썬 - 모스부호 클래스 만들기

```
class MorseCodeTranslator:
    def __init__(self):
        self.morse_dict = {
            "A": ".-",
            " ": " ",
        }
        self.english_dict = {value: key for key, value in self.morse_dict.items()}

    def english_to_morse(self, english_text):

    def morse_to_english(self, morse_text):

morse_translator = MorseCodeTranslator()
morse_text = morse_translator.english_to_morse("coding time")
english_text = morse_translator.morse_to_english(morse_text)
print("morse =", morse_text)
print("english = ", english_text)
```

### 3. 파이썬 - 모듈

1. 모듈 : 미리 작성된 함수 코드를 모아 놓은 파이썬 파일  
모듈화를 통해서 미리 구현된 다양한 라이브러리를 사용 가능

: 같은 폴더에 다음 코드를 작성

```
import my_module as mm  
  
print(mm.my_var)  
  
mm.my_func()  
  
mm.MyClass().work()
```

```
import my_module  
  
print(my_module.my_var)  
  
my_module.my_func()  
  
my_module.MyClass().work()
```

my\_module.py  
(파일이름이 모듈의 이름이 됩니다.)

```
my_var = "my_var"  
  
def my_func():  
    print("__name__ = ", __name__)  
  
class MyClass:  
    def work(self):  
        print("MyClass")
```

### 3. 파이썬 - 모듈

1. 모듈 : 미리 작성된 함수 코드를 모아 놓은 파이썬 파일  
모듈화를 통해서 미리 구현된 다양한 라이브러리를 사용 가능

: 같은 폴더에 다음 코드를 작성  
(사용할 클래스나 함수 나열)

```
from my_module import MyClass, my_var  
  
c = MyClass()  
c.work()  
  
print(my_var)
```

```
from my_module import *
```

...

: 전체 가져오기 (\*)

my\_module.py

```
my_var = "my_var"  
  
def my_func():  
    print("__name__ = ", __name__)  
  
class MyClass:  
    def work(self):  
        print("MyClass")
```



### 3. 파이썬 - 모듈

---

#### [문제]

MorseCodeTranslator 클래스를 morse.py 로 저장하고  
main.py 에서 morse 모듈을 사용하는 코드를 작성하세요

morse.py

```
class MorseCodeTranslator:
```

main.py

```
morse_translator = MorseCodeTranslator()  
english_text = morse_translator.morse_to_english(morse_text)  
print("english = ", english_text)
```

# 3. 파이썬 - 모듈

---

## [알아두기] 모듈 사용 방법

### 1) sys 모듈 사용하기

```
import sys
sys.path.append("c:/doit")  #모듈개발한 디렉토리추가

import mymod
```

### 2) PYTHONPATH 환경 변수 사용하기

```
$ set PYTHONPATH=c:/doit

import mymod
```

### 3) 현재 디렉토리 사용하기

### 3. 파이썬 - 라이브러리

1. 라이브러리 : 모듈과 모듈을 모아서 만든 개념. 외부에서 제공되는 모듈을 사용하려면 외부 라이브러리를 설치해야 함

1) requests - 웹에 데이터를 요청해서 가져오는 라이브러리

2) pip3 install requests (pip를 이용해서 requests라이브러리 설치)

3) 다음 파일을 저장하여 실행

```
C:\Users\LJH\AppData\Local\Programs\Python\Python37\Scripts>pip3 install requests
```

```
1 import requests
2 url = 'http://jspstudy.co.kr'
3 movie = requests.get(url)
4 print(movie.text)
```

```
<!DOCTYPE HTML>
<html lang="ko">
<head>
<meta charset="utf-8">
<meta http-equiv="X-UA-Compatible" content="IE=edge" />
<link rel="shortcut icon" href="favicon.ico">
<title>JSPStudy</title>
<!-- smartEditor시작 -->
<script type="text/javascript" src="../../se2/js/HuskyEZCreator.js" charset="utf-8">
</script>
<script type="text/javascript" src="../../se2/photo_uploader/plugin/hp_SE2M_AttachQuickPhoto.js" charset="utf-8"></script>
<!-- smartEditor끝 -->
```

# 3. 파이썬 - 라이브러리

---

## 2. 크롤링 (crawling)

1) 크롤링 혹은 스크래핑은 웹 페이지를 그대로 가져와서 필요한 데이터를 추출해 내는 행위

2) requests, BeautifulSoup 라이브러리 활용

: 라이브러리 설치

: pip3 install requests

: pip3 install beautifulsoup4

3) 참고 예제

: 네이버증권 주요뉴스 크롤링

: 크롤링할 주소 (URL) <http://finance.naver.com/>

# 3. 파이썬 - 라이브러리

## [따라하기] 네이버증권 주요뉴스 크롤링 (crawling)

```
import requests
from bs4 import BeautifulSoup

# URL of the finance section of the Naver website
url = "http://finance.naver.com/"

# Send a GET request to the URL
response = requests.get(url)

# Parse the HTML content using BeautifulSoup
soup = BeautifulSoup(response.content, "html.parser")

# Find the main section of the page
main_section = soup.find("div", {"class": "news_area"})
top_news_ul = main_section.find("ul")

# Find the list of top news articles
top_news = top_news_ul.find_all("span")

# Print the titles of the top news articles
for article in top_news:
    print(article.get_text())
```



The screenshot shows the Naver Finance homepage. At the top, there's a green header with 'NAVER 증권' and a search bar for '종목명·지수명 입력'. Below this is a navigation bar with links: '증권 홈', '국내증시', '해외증시', '시장지표', and '리서치'. A blue button labeled '최근조회종목' is visible, with 'MYSTOCK' below it. The '주요뉴스' (Main News) section is highlighted, showing a list of news items with titles like '중국 제조업 회복력 '기대이상'...', '1월 단꿈 2월 개꿈...3월 시계제로', and '[유림개장]장 초반 일제히 하락세...'. A link '더보기' (More) is at the end of the list.

# 3. 파이썬 - 라이브러리

Elements Console Sources Network >>

<div class="home\_spot view\_off">...</div>  
<div id="container" class="home">  
 <div id="content">  
 <div class="article">  
 <div class="section">  
 <div class="news\_area">  
 ...  
 <div class="section\_strategy"> == \$0  
 <h2 class="h\_strategy">  
 <span>주요뉴스</span>  
 </h2>  
 <ul>  
 <li>  
 <span>  
 <a href="/news/news\_read.naver?mode=mainnews&office\_id=008&article\_id=0004859839" onclick="clickcr(this, 'td n.list', '008\_0004859839', '0', event);">미국 '빅스텝' 공  
포 영향은 제각각...日증시는 되레 올랐네 [Asia마감]</a>  
 </span>  
 </li>  
 <li>...</li>  
 <li>...</li>  
 <li>...</li>  
 <li>...</li>  
 <li>...</li>  
 </ul>  
 </div>  
 </div>  
 </div>  
 </div>  
</div>

NAVER 증권

종목명·지수명 입력

증권 홈 국내증시 해외증시 시장지표 리서치

최근조회목록  
MYSTOCK

주요뉴스 > 더보기  
중국 제조업 회복력 '기대이상'...홍콩증시 4%대 '급등'[Asia마감]  
1월 단풍 2월 개꿈...3월 시계제로 [뉴욕마감]  
[유럽개장]장 초반 일제히 하락세... 프랑스 0.44%↓  
日 강보합...홍콩, 마스크 풀고도 하락 전환[Asia마감]  
[마켓뷰] 외국인 또 팔았지만 코스피 상승 마감... '테슬라 효과'...  
[마감시황]코스피, 기관 매수에 상승...2410선 마감

102